

Slide 1

MODULE-3

Slide 2

ADVERSARIAL SEARCH

Competitive environments, in which the agents' goals are in conflict, giving rise to adversarial search problems—often known as games.

Games, unlike most of the toy problems are interesting because they are too hard to solve.

Pruning allows us to ignore portions of the search tree that make no difference to the final choice, and heuristic evaluation functions allow us to approximate the true utility of a state without doing a complete search.

Slide 7

GAME SEARCH PROBLEM

Problem formulation

Initial state: initial board position + whose move it is

Operators: legal moves a player can make

Goal (terminal test): game over?

Utility (payoff) function: measures the outcome of the game and its desirability

Search objective:

Find the sequence of player's decisions (moves) maximizing its utility (payoff)

Consider the opponent's moves and their utility

Slide 8

GAME V/S SINGLE-AGENT SEARCH

We don't know how the opponent will act

The solution is not a fixed sequence of actions from start state to goal state, but a strategy or policy (a mapping from state to best move in that state)

Efficiency is critical to playing well

The time to make a move is limited

The branching factor, search depth, and number of terminal configurations are huge

In chess, branching factor ≈ 35 and depth ≈ 100 , giving a search tree of nodes

Number of atoms in the observable universe $\approx 10^{80}$

This rules out searching all the way to the end of the game

Slide 9

We first consider games with two players, max and min

MAX moves first, and then they take turns moving until the game is over.

At the end of the game, points are awarded to the winning player and penalties are given to the loser.

A game can be formally defined as a kind of search problem with the following elements:

S0: the initial state, which specifies how the game is set up at the start.

Player(s): defines which player has the move in a state.

Actions(s): returns the set of legal moves in a state.

Result(s, a): the transition model, which defines the result of a move.

terminal-test(s): A terminal test, which is true when the game is over and false otherwise. States where the games have ended are called terminal states

Utility(s, p): A utility function (also called an objective function or payoff function), defines the final numeric value for a game that ends in terminal state s for a player p. (In chess, the outcome is win, lose or a draw)

MIN MAX PROBLEM

Slide 10

The initial state, ACTIONS function, and RESULT function define the game tree for the game—a tree where the nodes are game states and the edges are moves.

From the initial state, MAX has nine possible moves.

Play alternates between max's placing an X and min's placing an O until we reach leaf nodes corresponding to terminal states such that one player has three in a row or all the squares are filled. The number on each leaf node indicates the utility value of the terminal state from the point of view of max; high values are assumed to be good for max and bad for min (which is how the players get their names)

Slide 13

Search tree is a tree that is superimposed on the full game tree, and examines enough nodes to allow a player to determine what move to make.

Slide 14

OPTIMAL SOLUTIONS IN GAMES

In a normal search problem, the optimal solution would be a sequence of actions leading to a goal state—a terminal state that is a win

In adversarial search, MIN has something to say STRATEGY about it.

MAX therefore must find a contingent strategy, which specifies max's move in the initial state, then max's moves in the states resulting from every possible response by MIN, then max's moves in the states resulting from every possible response by MIN to those moves, and so on

Slide 15

The possible moves for MAX at the root node are labelled a1, a2, and a3. The possible replies to a1 for MIN are b1, b2, b3, and so on. This particular game ends after one move each by MAX and MIN.

It is called the two-ply game.(consisting of two half moves)

Slide 16

MINIMAX VALUE

Given a game tree, the optimal strategy can be determined from the minimax value of each node, which we write as $\text{minimax}(n)$.

The minimax value of a node is of being useful in the corresponding state, assuming that both players play optimally from there to the end of the game.

The minimax value of a terminal state is just its utility(the state of being useful).

Max prefers to move to a state of maximum value, whereas min prefers a state of minimum value.

The terminal nodes on the bottom level get their utility values from the game's utility function

Minimax decision: optimal choice for max at root that leads to the state with the highest minimax value

Slide 18

In the min max algorithm in AI, there are two players, Maximiser and Minimiser.

Both these players play the game as one tries to get the highest score possible or the maximum benefit while the opponent tries to get the lowest score or the minimum benefit.

Every game board has an evaluation score assigned to it, so the Maximiser will select the maximised value, and the Minimiser will select the minimised value with counter moves.

If the Maximiser has the upper hand, then the board score will be a positive value, and if the Minimiser has the upper hand, then the board score will be a negative value.

This is based on the zero-sum game concept where the total utility score gets divided between the two players.

Thus, an increase in one player's score leads to a decrease in the opponent player's score, making the total score always zero. So, for one player to win, the other has to lose.

Slide 19

The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.

The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Keep on generating the game tree/ search tree till a limit d .

Compute the move using a heuristic function.

Propagate the values from the leaf node till the current position following the minimax strategy.

Make the best move from the choices.

Slide 21

For example, in the above figure, the two players MAX and MIN are there.

MAX starts the game by choosing one path and propagating all the nodes of that path.

Now, MAX will backtrack to the initial node and choose the best path where his utility value will be the maximum.

After this, it's MIN's chance. MIN will also propagate through a path and again will backtrack, but MIN will choose the path which could minimize MAX's winning chances or the utility value.

The time complexity of the minimax algorithm is $O(b^m)$.

The space complexity is $O(bm)$ for an algorithm that generates all actions at once, or $O(m)$ for an algorithm that generates actions one at a time

Slide 22

Minimax algorithm

Slide 23

ALPHA –BETA PRUNING

The problem with the minimax search is that the number of game states it has to examine is exponential in the depth of the tree.

We can eliminate or do pruning in the game tree

This particular technique is called alpha-beta pruning

It can be applied to trees of any depth, possible to prune entire subtrees rather than just leaves

Alpha- denotes the max value and beta –denotes the min value

Condition for Alpha-beta pruning:

The main condition which required for alpha-beta pruning is:

$$\alpha \geq \beta$$

Slide 25

Let the two unevaluated successors of node C have values x and y.

Then the value of the root node is given by

the value of the root and hence the minimax decision are independent of the values of the pruned leaves x and y

Slide 26

Alpha–beta pruning gets its name from the following two parameters that describe bounds on the backed-up values that appear anywhere along the path

Alpha–beta search updates the values of α and β as it goes along and prunes the remaining branches at a node (i.e., terminates the recursive call) as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.

Slide 28

EXAMPLE

Slide 29

MOVE ORDERING

The effectiveness of alpha–beta pruning is highly dependent on the order in which the states are examined

Eg, here we could not prune any successors of D at all because the worst successors (from the point of view of MIN) were generated first

If the third successor of D had been generated first, we would have been able to prune the other two.

It might be worthwhile to try to examine first the successors that are likely to be best

The best moves are often called killer moves and to try them first is called the killer move heuristic.

Slide 30

In many games, repeated states occur frequently because of transpositions

The hash table of previously seen positions is traditionally called a transposition table; it is essentially identical to the explored list in GRAPH-SEARCH

Slide 32

CONSTRAINT SATISFACTION PROBLEMS(CSP)

The search algorithms we discussed so far had no knowledge of the states representation (black box). - For each problem we had to design a new state representation (and embed in it the sub-routines we pass to the search algorithms).

Instead we can have a general state representation that works well for many different problems.

We can then build specialized search algorithms that operate efficiently on this general state representation.

Called as CSPs – constraint satisfaction problems.

Csp search algorithms take advantage of the structure of states and use general-purpose rather than problem-specific heuristics to enable the solution of complex problems.

The main idea is to eliminate large portions of the search space all at once by identifying variable/value combinations that violate the constraints

Slide 33

CSP

Slide 34

A constraint satisfaction problem (CSP) is a problem that requires its solution within some limitations or conditions also known as constraints. It consists of the following:

A finite set of variables which stores the solution ($V = \{V1, V2, V3, \dots, Vn\}$)

A set of discrete values known as domain from which the solution is picked ($D = \{D1, D2, D3, \dots, Dn\}$)

A finite set of constraints ($C = \{C1, C2, C3, \dots, Cn\}$)

all these sets should be finite except for the domain set. Each variable in the variable set can have different domains.

For example, consider the Sudoku problem again. Suppose that a row, column and block already have 3, 5 and 7 filled in. Then the domain for all the variables in that row, column and block will be $\{1, 2, 4, 6, 8, 9\}$.

Slide 35

WHY FORMULATE A PROBLEM AS A CSP?

CSP solvers can be faster than state-space searchers because the CSP solver can quickly eliminate large swatches of the search space.

One reason is that the CSPs yield a natural representation for a wide variety of problems.

With CSPs, once we find out that a partial assignment is not a solution, we can discard any further changes of that assignment.

we can see why the assignment is not a solution—

we see which variables violate a constraint—

so we can focus attention on the variables that matter.

As a result, many problems that are intractable for regular state-space search can be solved quickly when formulated as a CSP

Slide 36

EXAMPLE PROBLEM : MAP COLORING

Given a map of Australia, color it using three colors such that no neighboring territories have the same color.

Slide 37

To formulate this as a CSP,

we define the variables to be the regions $X = \{WA, NT, Q, NSW, V, SA, T\}$.

The domain of each variable is the set $D_i = \{\text{red, green, blue}\}$.

The constraints require neighboring regions to have distinct colors.

Since there are nine places where regions border, there are nine constraints:

$C = \{\text{SA} \neq \text{WA}, \text{SA} \neq \text{NT}, \text{SA} \neq \text{Q}, \text{SA} \neq \text{NSW}, \text{SA} \neq \text{V}, \text{WA} \neq \text{NT}, \text{NT} \neq \text{Q}, \text{Q} \neq \text{NSW}, \text{NSW} \neq \text{V}\}.$

There are many possible solutions to this problem, such as $\{\text{WA} = \text{red}, \text{NT} = \text{green}, \text{Q} = \text{red}, \text{NSW} = \text{green}, \text{V} = \text{red}, \text{SA} = \text{blue}, \text{T} = \text{red}\}.$

Slide 39

aaaaaaaaaaaaaaaaaaaaaaaaaaaaa

Search as State Space

Slide 40

To solve a CSP, we need to define a state space and the notion of a solution.

Each state in a CSP is defined by an assignment of values to some or all of the variables, $\{X_i = v_i, X_j = v_j, \dots\}$.

An assignment that does not violate any constraints is called a consistent or legal assignment.

A complete assignment is one in which every variable is assigned, and a solution to a CSP is a consistent, complete assignment.

A partial assignment is one that assigns values to only some of the variables

Slide 42

Precedence constraints

Whenever a task T1 must occur before task T2, and task T1 takes duration d1 to complete, we add an arithmetic constraint of the form

$$T1 + d1 \leq T2$$

Slide 43

A small part of the car assembly, consisting of 15 tasks: install axles (front and back), affix all four wheels (right and left, front and back), tighten nuts for each wheel, affix hubcaps, and inspect the final assembly. We can represent the tasks with 15 variables:

$X = \{\text{AxleF}, \text{AxleB}, \text{Wheel RF}, \text{Wheel LF}, \text{Wheel RB}, \text{Wheel LB}, \text{NutsRF}, \text{NutsLF}, \text{NutsRB}, \text{NutsLB}, \text{CapRF}, \text{CapLF}, \text{CapRB}, \text{CapLB}, \text{Inspect}\}.$

The value of each variable is the time that the task starts.

Slide 44

DISJUNCTIVE CONSTRAINT

Suppose we have four workers to install wheels, but they have to share one tool that helps put the axle in place.

We need a disjunctive constraint to say that AxleF and AxleB must not overlap in time; either one comes first or the other does:

$$(\text{AxleF} + 10 \leq \text{AxleB}) \text{ or}$$

$$(\text{AxleB} + 10 \leq \text{AxleF})$$

Slide 45

Constraints

For every variable except Inspect we add a constraint of the form $X + dX \leq \text{Inspect}$.

Finally, suppose there is a requirement to get the whole assembly done in 30 minutes.

We can achieve that by limiting the domain of all variables: $D_i = \{1, 2, 3, \dots, 27\}$.

This particular problem is trivial to solve, but CSPs have been applied to job-shop scheduling problems like this with thousands of variables.

Slide 48

Csps can be used to solve problems such as

Graph-coloring: given a graph, the a coloring of the graph means assigning each of its vertices a color such that no pair of vertices connected by an edge have the same color

In general, this is a very hard problem, e.g. Determining if a graph can be colored with 3 colors is np-hard

Many problems boil down to graph coloring, or related problems

Job scheduling: e.g. Suppose you need to complete a set of tasks, each of which have a duration, and constraints upon when they start and stop (e.g. Task c can't start until both task a and task b are finished)

Csps are a natural way to express such problems

Cryptarithmic puzzles: e.g. Suppose you are told that $TWO + TWO = FOUR$, and each of the letters corresponds to a different digit from 0 to 9, and that a number can't start with 0 (so T and F are not 0); what, if any, are the possible values for the letters?

While these are not directly useful problems, they are a simple test case for csp solvers

Slide 49

Types of Domains in CSP

There are following two types of domains which are used by the variables :

Discrete Domain: It is an infinite domain which can have one state for multiple variables. For example, a start state can be allocated infinite times for each variable.

Finite Domain: It is a finite domain which can have continuous states describing one domain for one specific variable. It is also called a continuous domain.

Eg. Map coloring, 8 Queens Pbm have Discrete, Finite domains

Infinite domains are set of integers or strings

Constraint Types in CSP

With respect to the variables, basically there are following types of constraints:

Unary Constraints: It is the simplest type of constraints that restricts the value of a single variable.

Binary Constraints: It is the constraint type which relates two variables. A value x_2 will contain a value which lies between x_1 and x_3 .

Global Constraints: It is the constraint type which involves an arbitrary number of variables. Eg. Alldiff in Cryptarithmic pbm

Slide 50

Some special types of solution algorithms are used to solve the following types of constraints:

Linear Constraints: These type of constraints are commonly used in linear programming where each variable containing an integer value exists in linear form only.

Non-linear Constraints: These type of constraints are used in non-linear programming where each variable (an integer value) exists in a non-linear form.

Slide 64

CONSTRAINT PROPAGATION

In local state-spaces, the choice is only one, i.e., to search for a solution. But in CSP, we have two choices either:

We can search for a solution or

We can perform a special type of inference called constraint propagation.

Constraint propagation is a special type of inference which helps in reducing the legal number of values for the variables.

The idea behind constraint propagation is local consistency.

The process of enforcing local consistency in each part of the graph causes inconsistent values to be eliminated throughout the graph.

Slide 65

It can be helpful to visualize a CSP as a constraint graph

The nodes of the graph correspond to variables of the problem,
and a link connects any two variables that participate in a constraint.

Slide 66

In local consistency, variables are treated as nodes, and each binary constraint is treated as an arc in the given problem.

There are following local consistencies which are discussed below:

Node Consistency: A single variable is said to be node consistent if all the values in the variable's domain satisfy the unary constraints on the variables. Eg:-SA $\neq$ green

where South Australians dislike green, the variable SA starts with domain {red, green, blue}, and we can make it node consistent by eliminating green, leaving SA with the reduced domain {red, blue}.

Arc Consistency: A variable is arc consistent if every value in its domain satisfies the binary constraints of the variables.

For example, consider the constraint $Y = X^2$ where the domain of both X and Y is the set of digits. We can write this constraint explicitly as $(X, Y), \{(0, 0), (1, 1), (2, 4), (3, 9)\}$.

To make X arc-consistent with respect to Y, we reduce X's domain to {0, 1, 2, 3}. If we also make Y arc-consistent with respect to X, then Y's domain becomes {0, 1, 4, 9} and the whole CSP is arc-consistent.

arc consistency can do nothing for the Australia map-coloring problem. Consider the following inequality constraint on (SA,WA): {(red, green),(red, blue),(green, red),(green, blue),(blue, red),(blue, green)}.

Slide 68

Arc Consistency detects failure earlier than Forward Checking

Slide 70

Arc consistency tightens down the domains (unary constraints) using the arcs (binary constraints).

To make progress on problems like map coloring, we need a stronger notion of consistency.

Path consistency tightens the binary constraints by using implicit constraints that are inferred by looking at triples of variables.

Path Consistency: When the evaluation of a set of two variable with respect to a third variable can be extended over another variable, satisfying all the binary constraints. It is similar to arc consistency.

A two-variable set $\{X_i, X_j\}$ is path-consistent with respect to a third variable X_m if, for every assignment $\{X_i = a, X_j = b\}$ consistent with the constraints on $\{X_i, X_j\}$, there is an assignment to X_m that satisfies the constraints on $\{X_i, X_m\}$ and $\{X_m, X_j\}$.

This is called path consistency because one can think of it as looking at a path from X_i to X_j with X_m in the middle.

Slide 71

k-consistency: This type of consistency is used to define the notion of stronger forms of propagation. Here, we examine the k-consistency of the variables.

Example :-

First, we choose a consistent value for X_1 .

We are then guaranteed to be able to choose a value for X_2 because the graph is 2-consistent, for X_3 because it is 3-consistent, and so on.

For each variable X_i , we need only search through the d values in the domain to find a value consistent with $X_1, \dots, X_{i-1}$. We are guaranteed to find a solution in time $O(n^2d)$.

Slide 79

Solutions are assignments satisfying all constraints, e.g.

$\{WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green\}$

Slide 80

SUDOKU EXAMPLE

Sudoku Playing: The gameplay where the constraint is that no number from 0-9 can be repeated in the same row or column.

Slide 81

n-queen problem: In n-queen problem, the constraint is that no queen should be placed either diagonally, in the same row or column.

Crossword: In crossword problem, the constraint is that there should be the correct formation of the words, and it should be meaningful.

Slide 82

CRYPTARITHMETIC PROBLEM:

Cryptarithmic Problem is a type of constraint satisfaction problem

where the game is about digits and its unique replacement either with alphabets or other symbols.

In cryptarithmic problem, the digits (0-9) get substituted by some possible alphabets or symbols.

The task in cryptarithmic problem is to substitute each digit with an alphabet to get the result arithmetically correct.

Slide 83

BACKTRACKING SEARCH

Chooses values for one variable at a time and backtracks when a variable has no legal values left to assign

Keeps only one single representation of a state and alters the representation ,rather than creating new state

The idea of choosing the variable with the fewest legal value is called minimum remaining values heuristics(MRV)

It has been called most constrained variable

MRV does not help in choosing the first region to color in Australia,because initially every region has three legal values

Degree heuristics is being considered

It reduces the branching factor by selecting the variable that is having largest number of constraints

After selecting the variable we have to decide the order in which to examine the values

Least constraining value heuristics is effective in these cases.
